

Math 116 Notes

Neil Makur

March 2021

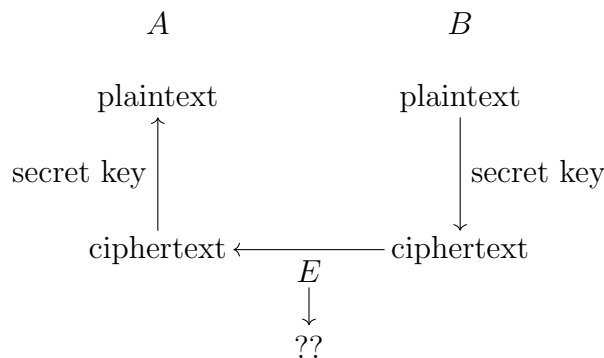
Contents

1	Lecture 1	3
2	Lecture 2	5
3	Lecture 3	6
4	Lecture 4	7
5	Lecture 5	8
6	Lecture 6	9
7	Lecture 7	10
8	Lecture 8	11
9	Lecture 9	12
10	Lecture 10	13
11	Lecture 11	14
12	Lecture 12	15
13	Lecture 13	16
14	Lecture 14	17
15	Lecture 15	18
16	Lecture 16	19
17	Lecture 17	20
18	Lecture 18	21

19 Lecture 19	22
20 Lecture 20	23
21 Lecture 21	24
22 Lecture 22	25
23 Lecture 23	26
24 Lecture 24	27
25 Lecture 25	29
26 Lecture 26	30
27 Lecture 27	31

1 Lecture 1

Cryptography is the study of hiding information. Suppose that we have two people, A and B that want to communicate in a way that nobody else can read. Say that B wants to send a message, which we call the plaintext, to A . After encrypting the plaintext, we get what is called the ciphertext. B would use a secret key to encrypt the plaintext. If A also has the secret key, then A can decrypt the ciphertext to get back the plaintext. However, if E intercepts the ciphertext, and does not have the secret key, then E cannot figure out the plaintext.



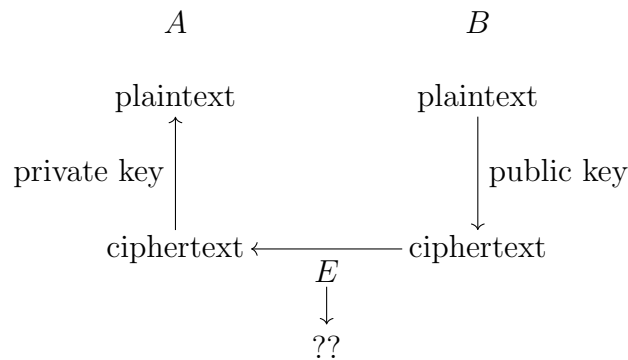
Definition 1.1. A Cryptosystem is the method to encrypt or decrypt a message given the secret key. In other words, it is an algorithm to compute the ciphertext given the plaintext and the secret key, and to compute the plaintext given the ciphertext and the secret key.

The security depends on the secrecy of both the secret key and the cryptosystem. We will generally write the plaintext in lowercase, and the ciphertext in upper case.

Example 1.2.

- (1) A Shift Cipher or a Caesar Cipher: If we have a plaintext (written in English), we shift the letters by some number. For instance, if we want to shift by 3, which in this case would be the secret key, ‘a’ would become ‘D’, ‘b’ becomes ‘E’, ‘c’ becomes ‘F’ and so on. So if we wanted to encrypt “hello” with this secret key, we would get “KHOOR”. However, this is not very secure, as there are only 26 possible shifts, so it is possible to try all keys (we call this type of attack an exhaustive search attack).
- (2) A Simple Substitution Cipher: We can encode our plaintext using an arbitrary choice for substitution. For example, we could send ‘a’ to ‘C’, ‘b’ to ‘B’, ‘c’ to ‘E’, ‘d’ to ‘A’, ‘e’ to ‘F’, and so on. In this case, the secret key would be the table containing what gets sent to what. In this case, there are $26! - 1$ possible codes, so an exhaustive search attack is not practical. However, this is still vulnerable. The letter ‘e’ appears the most in English words, so the most prominent letter likely encrypts ‘e’. After ‘e’, it is ‘t’, followed by ‘a’, and so on.
- (3) A Vignère Cipher: The secret key is a word. We write the word out repeatedly, and the letter of the alphabet in each position corresponds to the shift of that letter. For example, if the key is “bed”, then letters will be shifted by 2, 5, 4, 2, 5, 4, and so on.

Consider the scheme from the beginning of the section. This is what we call a symmetric cipher: both A and B have the secret key. However, there is the problem of A and B agreeing on the secret key. We fix this with the concept of an asymmetric cipher. In an asymmetric cipher, A has both a private key and a public key. If B (or anyone else) wants to send A a message, ey can use the public key to encrypt the message. After receiving the message, A uses the private key to decrypt the message.



It is also possible to use an asymmetric cipher as follows: A can sign a document using eir private key, and others can use their public key to verify the signature.

Symmetric ciphers are more efficient, so it is best to first use an asymmetric cipher to agree on the secret key to use in a symmetric cipher, and then to use a symmetric cipher.

An example of an asymmetric cipher is the RSA system. This uses the fact that large numbers are hard to factorize. In this case, the private key would be pq for large primes p and q , and the private key would be p and q independently. Then it is easy to construct the public key from the private key, but not the other way around.

2 Lecture 2

Recall that we write $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$.

Definition 2.1. Let $a, b, d \in \mathbb{Z}$. We say that a divides b , written $a|b$, if $b = ac$ for some $c \in \mathbb{Z}$. We say that d is a common divisor of a and b if $d|a$ and $d|b$. If d has the property that, for any common divisor e of a and b , $e|d$, we say that d is the greatest common divisor of a and b , denoted $\gcd(a, b)$.

Proposition 2.2. Let $a, b, q \in \mathbb{Z}$. Then $\gcd(a, b) = \gcd(aq + b, a)$.

Proof. Let $d = \gcd(a, b)$ and $d' = \gcd(aq + b, a)$. Then, $d|a$ and $d|b$, so $d|aq + b$ and $d|a$. Thus, $d|d'$. Now, $d'|aq + b$ and $d'|a$, so $d'|(aq + b - aq) = b$, so that $d'|d$. Thus, $d = d'$. ■

Note in particular, that if we write $a = bq + r$, then $\gcd(a, b) = \gcd(b, r)$.

Theorem 2.3. Let $a, b \in \mathbb{Z}$. There exist $u, v \in \mathbb{Z}$, so that $au + bv = \gcd(a, b)$.

This follows easily from repetition of the Euclidean Algorithm, keeping track of the coefficients.

3 Lecture 3

Definition 3.1. Let $m \in \mathbb{Z}_{>1}$, and $a, b \in \mathbb{Z}$. We say that a and b are congruent modulo m , denoted $a \equiv b \pmod{m}$, if $m \mid a - b$. In this case, m is called the modulus.

We define $\mathbb{Z}/m\mathbb{Z} = \{0, 1, \dots, m-1\}$, so any integer is congruent to an element of $\mathbb{Z}/m\mathbb{Z}$ modulo m . We can define $+$ and $\times$ on $\mathbb{Z}/m\mathbb{Z}$, by adding (multiplying) two elements and taking the unique element in $\mathbb{Z}/m\mathbb{Z}$ that is congruent to the result modulo m .

We call a set with $+$, $-$, and $\times$ (but not necessarily $\div$) with the associative and distributive laws is called a ring. We have seen that $\mathbb{Z}/m\mathbb{Z}$ is a ring, which we call the ring of integers modulo m . $\mathbb{Z}$ is also a ring, called the ring of integers. It is sometimes possible to divide by a particular element. For example, we can divide by 3 in $\mathbb{Z}/4\mathbb{Z}$ by multiplying by 3, as $3 \cdot 3 \equiv 1 \pmod{4}$. However, division by 2 does not make sense in $\mathbb{Z}/4\mathbb{Z}$, as there is no $a \in \mathbb{Z}/4\mathbb{Z}$ with $2a \equiv 1 \pmod{4}$.

Definition 3.2. Suppose that $a, b \in \mathbb{Z}/m\mathbb{Z}$. We say that b is the multiplicative inverse of a if $ab \equiv 1 \pmod{m}$. We write $b = a^{-1}$.

Proposition 3.3. Let $a \in \mathbb{Z}/m\mathbb{Z}$. Then a has an inverse if and only if $\gcd(a, m) = 1$.

Proof. Suppose that a has an inverse u modulo m . Then $au - 1 = mv$ for some $v \in \mathbb{Z}$. Thus, $1 = au - mv$. Since $\gcd(a, m) \mid a$ and $\gcd(a, m) \mid m$, we have that $\gcd(a, m) \mid au - mv = 1$, so $\gcd(a, m) = 1$. Now suppose that $\gcd(a, m) = 1$, and write $ab + km = 1$. Then, $ab - 1 = -km$, so $ab \equiv 1 \pmod{m}$. ■

If $a \in \mathbb{Z}/m\mathbb{Z}$, we say that a is a unit if it has an inverse. We denote the set of units by $(\mathbb{Z}/m\mathbb{Z})^\times$. We call a set with an operation and the inverse operation a group.

Definition 3.4. The Euler Totient Function is defined by $\phi(m) = |(\mathbb{Z}/m\mathbb{Z})^\times|$.

If m has prime factorization $p_1^{n_1} \cdots p_r^{n_r}$, then

$$\phi(m) = \prod_{k=1}^r p_k^{n_k-1} (p_k - 1) = m \prod_{k=1}^r \left(1 - \frac{1}{p_k}\right).$$

Consider the shift Cipher. The plaintext is an element $P \in \mathbb{Z}/26\mathbb{Z}$, the key is an element $k \in \mathbb{Z}/26\mathbb{Z}$, and the ciphertext is an element $C \in \mathbb{Z}/26\mathbb{Z}$. Encryption is given by $C = P + k$, and decryption is given by $P = C - k$.

4 Lecture 4

We call a ring where we have a $\div$ operation (as long as we are not dividing by zero) a field. For instance, $\mathbb{Z}/p\mathbb{Z}$ is a field for prime p , as every nonzero element of $\mathbb{Z}/p\mathbb{Z}$ is coprime to p , and thus has an inverse. We write $\mathbb{F}_p$ for this field.

Theorem 4.1 (Fundamental Theorem of Arithmetic). *Suppose that $n \in \mathbb{Z}_{\geq 2}$. Then we can write*

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r},$$

where the p_i 's are distinct primes, and $e_i \in \mathbb{Z}_{\geq 1}$ for all i .

Definition 4.2. The order of n at p , denoted $\text{ord}_p(n)$ is the smallest e such that $p^e | n$ but $p^{e+1} \nmid n$.

Let us consider powers a^n in $\mathbb{F}_5$. We have that

$a \backslash n$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
2	1	2	4	3	1	2	4
3	1	3	4	2	1	3	4
4	1	4	1	4	1	4	1

For all $a \in \mathbb{F}_5^\times$, $a^4 = 1$. This is true generally.

Theorem 4.3 (Fermat's Little Theorem). *For all $a \in (\mathbb{Z}/p\mathbb{Z})^\times$, $a^{p-1} \equiv 1 \pmod{p}$. More generally, for all $a \in (\mathbb{Z}/m\mathbb{Z})^\times$, $a^{\phi(m)} \equiv 1 \pmod{m}$.*

Let us consider how to find $a^{-1} \pmod{p}$. Using the Euclidean Algorithm, keeping track of the numbers, we can find u and v such that $au + pv = 1$. Then $a \cdot u \equiv 1 \pmod{p}$, so $u = a^{-1}$ in $(\mathbb{Z}/p\mathbb{Z})^\times$. We can also use Fermat's Little Theorem to get that $a \cdot a^{p-2} \equiv 1 \pmod{p}$, so that $a^{-1} \equiv a^{p-2} \pmod{p}$.

Definition 4.4. For $a \in (\mathbb{Z}/p\mathbb{Z})^\times$, the order of a , denoted $\text{ord}(a)$, is the value $\min\{k \in \mathbb{Z}_{\geq 1} : a^k \equiv 1 \pmod{p}\}$.

For instance, in $(\mathbb{Z}/5\mathbb{Z})^\times$, $\text{ord}(1) = 1$, $\text{ord}(2) = 4$, $\text{ord}(3) = 4$ and $\text{ord}(4) = 2$.

Let us consider the question if it is possible for $\text{ord}(a) < p-1$ for all $a \in (\mathbb{Z}/p\mathbb{Z})^\times$. This is impossible by the Primitive Root Theorem.

Theorem 4.5 (Primitive Root Theorem). *There exists $a \in \mathbb{F}_p^\times$ such that $\text{ord}(a) = p-1$. In particular, $\{a^0, a^1, \dots, a^{p-1}\} = \mathbb{F}_p^\times$. We call this a a primitive root or generator of $\mathbb{F}_p^\times$.*

5 Lecture 5

Definition 5.1. A cryptosystem is a tuple $(\mathcal{K}, \mathcal{M}, \mathcal{C}, e, d)$, where $\mathcal{K}$ is the set of possible keys, $\mathcal{M}$ is the set of possible plaintexts, $\mathcal{C}$ is the set of possible ciphertexts, $e : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$ is an encryption function, and $d : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$ is a decryption function. For $k \in \mathcal{K}$, we write $e_k : \mathcal{M} \rightarrow \mathcal{C}$ for the function $e_k(m) = e(k, m)$ and $d_k : \mathcal{C} \rightarrow \mathcal{M}$ for the function $d_k(c) = d(k, c)$. We also require that d_k and e_k are inverses for all $k \in \mathcal{K}$.

We would also like e and d to be easy to compute, and that it would be hard to compute plaintext from the ciphertext without the k (i.e. given $c_1, \dots, c_n$, it is hard to compute $d_k(c_1), \dots, d_k(c_n)$ without knowing k). We would further like that, given $(m_1, c_1), \dots, (m_n, c_n) \in \mathcal{M} \times \mathcal{C}$, where $c_i = e_k(m_i)$ for some fixed $k \in \mathcal{K}$, it is still hard to compute $d_k(c)$ if $c \neq c_i$ for all i . Finally, we would also like that, for any chosen $m_1, \dots, m_n \in \mathcal{M}$ and $c_i = e_k(m_i)$, it is still hard to compute $d_k(c)$ with $c \neq c_i$.

For instance, the substitution cipher does not satisfy the last requirement, as we can just choose m_i to be the i th letter for $1 \leq i \leq 26$.

Let us look at some symmetric ciphers.

Pick a prime number $p \sim 2^{160}$, which is public information. We take $\mathcal{K} = \mathbb{Z}/p\mathbb{Z}$, and $\mathcal{M} = \mathcal{C} = \mathbb{Z}/p\mathbb{Z}$. We define $e_k(m) = m + k$, and $d_k(c) = c - k$. In this, e and d are easy to compute, and without k it is hard to compute d_k . However, if it is known that $c = e_k(m)$, we can solve for k and find d_k .

Once again, pick prime $p \sim 2^{160}$, and let $\mathcal{K} = \mathcal{M} = \mathcal{C} = (\mathbb{Z}/p\mathbb{Z})^\times$. We define $e_k(m) = km$ and $d_k(c) = k^{-1}c$. Given k , e and d are easy to compute, and without knowing k , it is hard to compute d_k . However, if it is known that $e_k(m) = c$, then it is easy to compute k .

Pick prime $p \sim 2^{160}$. We take $\mathcal{K} = (\mathbb{Z}/p\mathbb{Z})^\times \times (\mathbb{Z}/p\mathbb{Z})$, and $\mathcal{M} = \mathcal{C} = \mathbb{Z}/p\mathbb{Z}$. Write $k = (k_1, k_2) \in \mathcal{K}$. We define $e_k(m) = k_1 m + k_2$, and $d_k(c) = k_1^{-1}(c - k_2)$. Given k , it is easy to compute e and d , and without k it is hard to compute d_k . However, if we know that $c_1 = e_k(m_1)$ and $c_2 = e_k(m_2)$, then we can solve for k .

Take $p \sim 2^{160}$ to be prime. Set $\mathcal{K} = \text{GL}_n(\mathbb{Z}/p\mathbb{Z}) \times (\mathbb{Z}/p\mathbb{Z})^n$, where $\text{GL}_n(F)$ denotes the set of invertible $n \times n$ matrices with entries in F . We set $\mathcal{M} = \mathcal{C} = (\mathbb{Z}/p\mathbb{Z})^n$. We define $e_{(k_1, k_2)}(m) = k_1 m + k_2$ and $d_{(k_1, k_2)}(c) = k_1^{-1}(c - k_2)$. It is easy to compute e and d , and hard to compute d_k without k . However, if $c_i = e_k(m_i)$ for $1 \leq i \leq n+1$, then we can solve for k , assuming that $\{m_i - m_{n+1} : 1 \leq i \leq n\}$ forms a basis for $(\mathbb{Z}/p\mathbb{Z})^n$.

6 Lecture 6

Suppose that $g \in \mathbb{F}_p^\times$ is a generator, so that $\mathbb{F}_p^\times = \{g^0, g^1, \dots, g^{p-2}\}$, and $h \in \mathbb{F}_p^\times$. The discrete log problem (DLP) is finding $0 \leq x < p-1$ such that $g^x = h$. We denote such an x by $\log_g h$. For example, let us find $\log_2 7$ in $\mathbb{F}_{11}$. We have that $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 5$, $2^5 = 10$, $2^6 = 9$, and $2^7 = 7$, so $\log_2 7 = 7$.

Notice that DLP still makes sense in any group G . For example, in $G = \mathbb{Z}/p\mathbb{Z}$, we essentially wish to find the value x such that $\underbrace{g + \dots + g}_{x \text{ times}} = h$, so $x = g^{-1}h$.

We discuss the Diffie-Hellman key exchange. Suppose that A and B want to decide a secret key k to use later. Take p to be a large prime, and g to be a generator of $\mathbb{F}_p$. A will pick $a \in \mathbb{Z}/(p-1)\mathbb{Z}$, and B will pick $b \in \mathbb{Z}/(p-1)\mathbb{Z}$. Then, A will compute $\mathcal{A} = g^a$ and B will compute $\mathcal{B} = g^b$. A will then give B $\mathcal{A}$, and B will give A $\mathcal{B}$. Then A will compute $\mathcal{B}^a$, and B will compute $\mathcal{A}^b$. These are both equal to g^{ab} , and is what their secret key will be. This allows A and B to agree on a number, but not to send messages to each other.

Suppose that E is able to obtain g^a and g^b , and wants to find g^{ab} . If E can solve DLP, then E can find g^{ab} . We call this specific problem the Diffie-Hellman Problem (DHP), so that DLP is harder than DHP. We call the problem “given g^a, g^b, C , is $g^{ab} = C$?” the Decision Diffie-Hellman Problem (DDH). Then DHP is harder than DDH.

We now discuss the Elgamal Public Key Cryptosystem (PKC). A will have a private key $a \in \mathbb{Z}/(p-1)\mathbb{Z}$, and the public key is $\mathcal{A} = g^a \in \mathbb{F}_p^\times$. If B wants to send a message with plaintext m to A , then B picks a random $k \in \mathbb{Z}/(p-1)\mathbb{Z}$, and computes $c_1 = g^k$ and $c_2 = m \cdot \mathcal{A}^k$, and send these to A . A will then compute $c_2 c_1^{-a} = c_2 g^{-ak} = c_2 \mathcal{A}^{-k} = m$.

7 Lecture 7

We reduce solving the Elgamal problem to solving DHP. Suppose that you know $c_1 = g^k$, $c_2 = m \cdot \mathcal{A}^k$, and $\mathcal{A} = g^a$. If we can find g^{ak} , then we can find $c_2 g^{-ak} = m$, and g^{ak} is DHP on c_1 and $\mathcal{A}$.

Now, suppose that we can solve the Elgamal problem. Given g^a and g^b , let $A = g^a$, $c_1 = g^b$, and $c_2 = 1$. Solving the Elgamal problem, we get $c_2 c_1^{-a} = g^{-ab}$, so we can find g^{ab} .

Let us look at the brute force approach of solving DLP. If G is a group, g generates G , and $h \in G$, we can compute $g^0, g^1, \dots, g^{|G|-1}$ until we get h . Computing g^x needs about $\log_2 x$ multiplications, and we need to compute $|G|$ of these, we there are about $|G| \log_2 x$ operations. Thus, if each x and $|G|$ have k bits, we need $k \cdot 2^k$ operations.

8 Lecture 8

Definition 8.1. Let $f, g : \mathbb{R} \rightarrow \mathbb{R}^+$. We write $f(x) = \mathcal{O}(g(x))$ if there is some constant C and d such that $\frac{f(x)}{g(x)} \leq C$ whenever $x \geq d$.

Note that if $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)}$ exists, and is not infinite, then $f(x) = \mathcal{O}(g(x))$.

For example,

$$\lim_{x \rightarrow \infty} \frac{x^n}{2^x} = 0,$$

so $x^n = \mathcal{O}(2^x)$. However,

$$\lim_{x \rightarrow \infty} \frac{2^x}{x^n} = \infty,$$

so $2^x \neq \mathcal{O}(x^n)$. As another example, $\frac{\sin(x)}{1} \leq 1$ for all x , so $\sin(x) = \mathcal{O}(1)$.

If $f(x) = \mathcal{O}(2^x)$, we say that $f(x)$ takes exponential time. If $f(x) = \mathcal{O}(2^{\varepsilon x})$ for all $\varepsilon > 0$, we say that f takes subexponential time, and if $f(x) = \mathcal{O}(x^n)$ for some n , we say that f takes polynomial time.

When solving DLP, we need at most $n - 1$ multiplication operations. If $g \in \mathbb{F}_p^\times$ is a generator, then $n = p - 1$, so we need $p - 2$ operations. Thus if p has k bits, we need about 2^k , so we need $f(k) = \mathcal{O}(2^k)$ operations. Now, if G is a group of order N , and g is a generator of G and $h \in G$, let us look at an algorithm to solve for x such that $g^x = h$. Set $n = \lfloor \sqrt{N} \rfloor + 1$ and create two lists: $g_0, g_1, \dots, g^{n-1}$ and $h, h \cdot g^{-n}, \dots, h \cdot g^{-(n-1)n}$. Now, find an intersection between the two lists, which must happen by the division algorithm. We then have $g^i = h \cdot g^{-jn}$, so that $h = g^{nj+i}$. Now, creating the two lists both have $\mathcal{O}(n)$, and the sorting and searching algorithm is $\mathcal{O}(n \log n)$, so overall the solution is $\mathcal{O}(n \log n) = \mathcal{O}(\sqrt{N} \log \sqrt{N})$. If N has k bits, then this is $\mathcal{O}(k/2 \cdot 2^{k/2}) = \mathcal{O}(k \cdot 2^{k/2})$. This is better than brute force, but still exponential time.

9 Lecture 9

Theorem 9.1 (Chinese Remainder Theorem). *Let $m_1, \dots, m_r$ be pairwise relatively prime integers. Let $a_1, \dots, a_r$ be integers. Then there exists a unique $0 \leq x < m_1 m_2 \cdots m_r$ satisfying $x \equiv a_i \pmod{m_i}$.*

We prove this with an algorithm demonstrated with the case $1000 \leq x \leq 1100$, $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$. Write $x_1 \equiv 2 \pmod{3}$. Then, we have that $x_2 = 2 + 3y_2 \pmod{3 \cdot 5}$. Now, $y_2 \equiv (3 - 2) \cdot 3^{-1} \equiv 2 \pmod{5}$. Thus, $x \equiv 8 \pmod{15}$. Now, we have $x_3 \equiv 8 + 15y_3 \pmod{15 \cdot 7}$, so that $2 \equiv 8 + 15y_3 \pmod{7}$. Thus, $y_3 = (2 - 8) \cdot 15^{-1} \equiv 1 \pmod{7}$. Thus, $x_3 = 8 + 15 = 23 \pmod{105}$. Now, we want $1000 \leq x \leq 1100$, so we add 1050 to get 1073.

Suppose that g is a generator of $(\mathbb{Z}/(N+1)\mathbb{Z})^\times$, where $N+1$ is prime, and that we want to solve DLP with h . Write $N = p_1^{e_1} \cdots p_r^{e_r}$, and let $g_i = g^{Np_i^{-e_i}}$ (note that $\text{ord}(g_i) = p_i^{e_i}$) and $h_i = h^{Np_i^{-e_i}}$. Solve for x_i so that $g_i^{x_i} \equiv h_i \pmod{p_i^{e_i}}$. Using the Chinese Remainder Theorem, solve for $x \equiv x_i \pmod{p_i^{e_i}}$ to obtain $x \pmod{N}$. We claim that x is a solution to DLP. Suppose that x_0 is a solution to $g^{x_0} = h$, and let x_i be as above. We want to show that $x_0 \equiv x_i \pmod{p_i^{e_i}}$. We have that $g^{x_0} = h$, so that $g_i^{x_0} = g^{x_0 \cdot Np_i^{-e_i}} = h^{Np_i^{-e_i}} = h_i$. Since $\text{ord}(g_i) = p_i^{e_i}$, we have that $x_0 \equiv x_i \pmod{p_i^{e_i}}$.

Now suppose that g is a generator of $(\mathbb{Z}/(p^e+1)\mathbb{Z})^\times$. Then $g^{p^{e-1}}$ has order p . If we solve

$$(g^{p^{e-1}})^{x_0} = h^{p^{e-1}} \quad \text{and} \quad (g^{p^{e-1}})^{x_1} = (hg^{-x_0})^{p^{e-2}},$$

then $x \equiv x_0 \pmod{p}$, and the second term gives that $g^{p^{e-2}(x_0+x_1p)} = g^{x_0p^{e-2}+x_1p^{e-1}} = h^{p^{e-2}}$, so that $x \equiv x_0 + x_1 \pmod{p^2}$. Continuing this process, we can solve for $x \pmod{p^3}$, and so on. Thus, we have reduced the general DLP to the case when N is a prime.

The lesson of this is that we should not allow $p-1$ to be the product of small primes when needing DLP to be secure.

10 Lecture 10

We discuss the Pohlig-Hellman algorithm. Suppose that we wanted to find x such that $5^x = 4$ in $\mathbb{F}_7^\times$. Let $g = 5$ so that $\text{ord}(g) = 6$. Let $g_1 = 5^{6/3} = 5^3 \equiv -1 \pmod{7}$. Then, $\text{ord}(g_1) = 2$. Now, if $g_2 = 5^{6/3} = 5^2 \equiv 4 \pmod{7}$, then $\text{ord}(g_2) = 3$. Now let $h = 4$, and $h_1 = 4^{6/2} = 4^3 \equiv 1 \pmod{7}$. Take $h_2 = 4^{6/3} = 4^2 \equiv 2 \pmod{7}$. We solve $g_1^{x_1} \equiv h_1$, so that $x_1 \equiv 0 \pmod{2}$, and $g_2^{x_2} \equiv h_2$, so that $x_2 \equiv 2 \pmod{3}$. Now, the corresponding x is $2 \pmod{6}$, so this is our answer.

11 Lecture 11

Proposition 11.1. *Suppose that $\gcd(e, p-1) = 1$. Then there exists a unique $x \in \mathbb{F}_p^\times$ such that $x^e \equiv h \pmod{p}$, for some fixed h .*

Proposition 11.2. *Write $eu + (p-1)v = 1$. Now, $x = x^{eu+(p-1)v} \equiv x^{eu} \pmod{p}$. Thus, if such an x exists, then it must be h^u . Now, we have that*

$$(h^u)^e \equiv h^{eu} \equiv h^{eu+(p-1)v} \equiv h,$$

so we have existence.

Proposition 11.3. *If $\gcd(e, \phi(N)) = 1$, then there exists a unique $x \in (\mathbb{Z}/N\mathbb{Z})^\times$ such that $x^e \equiv h \pmod{N}$ for some fixed $h \in (\mathbb{Z}/N\mathbb{Z})^\times$.*

The proof of this is the same as in the previous case.

Now, consider the RSA public key cipher. For primes p and q , we choose e with $\gcd(e, (p-1)(q-1)) = 1$. The private key is p and q , while the public key is $N = pq$. Given plaintext m , the cipher text is $c = m^e \pmod{N}$, and this is possible to decrypt given p and q .

12 Lecture 12

Suppose that there is $a \in \mathbb{Z} \setminus \{0\}$ with $a^{N-1} \not\equiv 1 \pmod{N}$ (or an $a \in \mathbb{Z}$ with $a^N \not\equiv 1 \pmod{N}$). Then, we must have that N is composite. However, there are composite numbers with $a^N \equiv a \pmod{N}$ for all $a \in \mathbb{Z}$. For example, $\phi(560) = 80 \mid 560 - 561 - 1$, so 561 works.

Suppose that p is prime and that $p - 1 = 2^k q$ for odd q . Then either $a^q \equiv 1 \pmod{p}$ or $a^{2^i q} \equiv -1 \pmod{p}$ for some $0 \leq i \leq k-1$ (this follows from the fact that $a^{2^k q} \equiv 1 \pmod{p}$). Thus, if there is an a to contradict this, N is composite.

13 Lecture 13

Suppose that $N = pq$, and write $p - 1 = p_1^{e_1} \cdots p_r^{e_r}$. Then, for all $a \in (\mathbb{Z}/N\mathbb{Z})^\times$, we have that $1 \equiv a^{p-1} \equiv a^{p_1^{e_1} \cdots p_r^{e_r}} \pmod{p}$. Now, $p - 1 | n!$ for large n . Thus, for large n , we have that $1 \equiv a^{n!} \pmod{p}$. Now, consider $\gcd(a^{n!} - 1, N)$. We can probably get p from the gcd here. If we compute this sequentially for $n = 1, 2, 3, \dots$ until the gcd is not 1, the gcd is probably p .

14 Lecture 14

Recall that if $N = a^2 - b^2$, then $N = (a + b)(a - b)$. Equivalently, if $a^2 \equiv b^2 \pmod{N}$, then $(a + b)(a - b) \equiv 0 \pmod{N}$, so (if $N = pq$) $\gcd(a - b, N) = p, q$ or N , so we may be able to factor N . We describe a process to attempt to factor N .

First, find $\sqrt{N} < a_1, \dots, a_r < \sqrt{2N}$, so that $a_i^2 - N$ has “small” prime factors (i.e. all prime factors are less than some value B). Now, we find $x_1, \dots, x_r$ equal to 0 or 1 such that $c_1^{x_1} \cdots c_r^{x_r}$ is a square, where $c_i = a_i^2 - N = p_1^{e_{i,1}} \cdots p_s^{e_{i,s}}$ (where $p_1, \dots, p_s$ are primes less than B). Now, $(a_1^{x_1} \cdots a_r^{x_r})^2 \equiv c_1^{x_1} \cdots c_r^{x_r} \pmod{N}$, where the right hand side is a different square, so we can consider the gcd of their difference.

15 Lecture 15

Suppose that $N = pq$, and suppose we want to determine whether a is a square $\pmod N$. Now, we would have that A has a private key p, q and a public key $N = pq$ and a with $a \not\equiv u^2 \pmod{pq}$. So, if B wanted to send $m = 0, 1$, they would choose a random $r \in \mathbb{Z}/N\mathbb{Z}$ and get $c = r^2$ if $m = 0$ and ar^2 if $m = 1$. Then, if c is a square $\pmod p$, $m = 0$, and if c is not a square $\pmod p$, then $m = 1$.

We say that $a \in \mathbb{Z}/N\mathbb{Z}$ is a quadratic residue if a is a square $\pmod N$.

If $N = p$ is an odd prime, then the product of a quadratic residue with another quadratic residue is a quadratic residue, the product of a quadratic residue with a non-quadratic residue is not a quadratic residue, and the product of two non-quadratic residues is a quadratic residue. To see the last statement, take $g \in (\mathbb{Z}/p\mathbb{Z})^\times$ to be a primitive root, so that a non-quadratic residue must be an odd power of g . Then the product of these must be an even power of g .

Definition 15.1. We define the Legendre Symbol

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & a \equiv u^2 \pmod{p} \\ -1 & a \not\equiv u^2 \pmod{p} \\ 0 & p|a \end{cases},$$

where p is an odd prime and $a \in \mathbb{Z}$.

Thus, we have that

$$\left(\frac{a}{p}\right) \left(\frac{b}{p}\right) = \left(\frac{ab}{p}\right).$$

Theorem 15.2 (Quadratic Reciprocity). *Let p be an odd prime.*

$$(1) \quad \left(\frac{-1}{p}\right) = \begin{cases} 1 & p \equiv 1 \pmod{4} \\ -1 & p \equiv 3 \pmod{4} \end{cases}.$$

$$(2) \quad \left(\frac{2}{p}\right) = \begin{cases} 1 & p \equiv 1, 7 \pmod{8} \\ -1 & p \equiv 3, 5 \pmod{8} \end{cases}.$$

$$(3) \quad \text{Let } q \text{ be an odd prime. Then } \left(\frac{p}{q}\right) \left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2} \cdot \frac{q-1}{2}}.$$

For example, let us find $\left(\frac{21}{53}\right)$. We have that $\left(\frac{21}{53}\right) = \left(\frac{3}{53}\right) \left(\frac{7}{53}\right)$. Now, $\left(\frac{3}{53}\right) = (-1)^{(3-1)(53-1)/4} \left(\frac{53}{3}\right) = \left(\frac{53}{3}\right) = \left(\frac{2}{3}\right) = -1$ and $\left(\frac{7}{53}\right) = (-1)^{(7-1)(53-1)/4} \left(\frac{53}{7}\right) = \left(\frac{53}{7}\right) = \left(\frac{4}{7}\right) = 1$. Thus, we have that $\left(\frac{21}{53}\right) = -1$.

If $N = p_1^{e_1} \cdots p_r^{e_r}$, we write

$$\left(\frac{a}{N}\right) = \left(\frac{a}{p_1}\right)^{e_1} \cdots \left(\frac{a}{p_r}\right)^{e_r}.$$

16 Lecture 16

If a is a quadratic residue mod N , then $\left(\frac{a}{N}\right) = 1$. However, for nonprime N , if $\left(\frac{a}{N}\right) = 1$, we are not guaranteed that a is a quadratic residue.

We discuss the Golwasser-Micali PKC. The private key will be p and q , while the public key will be $N = pq$ and $a \in (\mathbb{Z}/N\mathbb{Z})^\times$ $\left(\frac{a}{p}\right) = \left(\frac{a}{q}\right) = -1$. If the plaintext is $m = 0$ or 1, we choose a random r , and the ciphertext c is $c = r^2$ if $m = 0$ and ar^2 if $m = 1$. Then $\left(\frac{c}{p}\right) = (-1)^m$, but $\left(\frac{c}{N}\right) = 1$.

We now discuss how to have digital signatures. We will have that A has a private key, which we call a signing key. We also call the public key the verification key. Using the signing key, A can sign a document D with a signature S . Using the verification key, B can determine from (D, S) whether it is signed by A or not.

For example in the RSA digital signature, the signing key are primes p and q , and d such that $de \equiv 1 \pmod{\phi(N)}$ (see e next), and the verification key is $N = pq$ and e such that $\gcd(e, \phi(N)) = 1$. For $D \in (\mathbb{Z}/N\mathbb{Z})^\times$, we compute $S \equiv D^d \pmod{N}$. To verify, we have that $S^e = D$. Now, to forge a signature, we compute $S \equiv D^{1/e} \pmod{N}$.

In the Elgamal digital signature, with a prime p and a primitive root g of $(\mathbb{Z}/p\mathbb{Z})^\times$, the signing key is $a \in \mathbb{Z}/(p-1)\mathbb{Z}$, and the verification key is $A = g^a \in (\mathbb{Z}/p\mathbb{Z})^\times$. For a random $k \in \mathbb{Z}/(p-1)\mathbb{Z}$, we have $S_1 = g^k \in (\mathbb{Z}/p\mathbb{Z})^\times$ and $S_2 = (D - aS_1)k^{-1} \pmod{p-1}$. We can then verify that $S_1^{S_2} A^{S_1} \equiv g^D \pmod{p}$. To forge a signature, we find S_1 and S_2 so that $S_1^{S_2} A^{S_1} \equiv g^D \pmod{p}$.

The digital signature algorithm (DSA) is an improvement on efficiency for the Elgamal. We pick a prime $q|p-1$, and $g_q \in (\mathbb{Z}/p\mathbb{Z})^\times$ to have order q . The signing key is $a \in \mathbb{Z}/p\mathbb{Z}$, and the verification key is $A = g_q^a \in (\mathbb{Z}/p\mathbb{Z})^\times$. We choose a random $k \in \mathbb{Z}/q\mathbb{Z}$, and suppose that $D \in \mathbb{Z}/q\mathbb{Z}$, and compute $S_1 = (g_q^k \pmod{p}) \pmod{q}$ and $S_2 = (D + aS_1)k^{-1} \pmod{q}$. We then have that $g_q^{DS_2^{-1} A^{S_1 S_2^{-1}}} \equiv S_1 \pmod{q}$.

17 Lecture 17

An Elliptic Curve is a curve on the $x - y$ plane defined by $y^2 = x^3 + Ax + B$, with $4A^3 + 27B^2 \neq 0$. We can “add” points on Elliptic Curves. To add (x_1, y_1) and (x_2, y_2) , we draw the line passing through them, and find its remaining intersection with the curve. The sum is then defined to be the reflection of the intersection point about the x -axis. If we wished to find an identity element, the identity would be the point at $(0, \infty)$, as this creates a vertical line, meaning that the sum of this with a point P would be P . We thus amend our definition as follows.

Definition 17.1. An elliptic curve E is the curve $y^2 = x^3 + Ax + B \cup \{(0, \infty)\}$.

We denote this $(0, \infty)$ point by 0 . Then, E is a group under the addition described above. Note that we can actually use any curve $y^2 = x^3 + ax^2 + bx + c$, as it can be expressed in the form $y^2 = (x - d)^3 + A(x - d) + B$.

For example, let us add $(0, 0)$ and $(3, 6)$ on the curve $y^2 = x(x - 1)(x + 3)$. We have that the line passing through these points is $y = 2x$, so we wish to solve $(2x)^2 = x^3 + 2x^2 - 3x$, or $x^3 - 2x^2 - 3x = 0$. This is $x(x - 3)(x + 1) = 0$, so we take the remaining x -coordinate, which is $x = -1$. Thus, the intersection is $(-1, -2)$. Reflecting, the sum is $(-1, 2)$.

As another example, let us find $(3, 6) + (3, 6)$. We have that $y^2 = x^3 + 2x^2 - 3x$, so that $2yy' = 3x^2 + 4x - 3$. Thus, at $(3, 6)$, the slope is 3 , so that the tangent line at $(3, 6)$ is $y - 6 = 3(x - 3)$. The remaining intersection point is $(1, 0)$. Thus, the sum is $(1, -0) = (1, 0)$.

18 Lecture 18

The term elliptic curve comes from the elliptic integral, which is finding the circumference of ellipses. The circumference of $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ is

$$2 \int_{-a}^a \sqrt{\frac{a^2 - (1 - b^2/a^2)x^2}{a^2 - x^2}} dx.$$

If $k^2 = 1 - b^2/a^2$ and we write $x = at$, we get that this is

$$2a \int_{-1}^1 \sqrt{\frac{1 - k^2 t^2}{1 - t^2}} dt.$$

We write $\mathcal{O}$ for the infinity point. We consider

$$E(\mathbb{F}_p) = \mathcal{O} \cup \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + Ax + B, 4A^3 + 27B^2 \neq 0\},$$

which forms a (finite) group. For example, when E is $y^2 = x(x-1)(x+3)$, we have that

$$E(\mathbb{F}_5) = \{\mathcal{O}, (0, 0), (1, 0), (2, 0), (3, 1), (3, 4), (4, 2), (4, 3)\}.$$

19 Lecture 19

We can compute the group $E(\mathbb{F}_5)$ as above, and get $E(\mathbb{F}_5) \cong \mathbb{Z}/4\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$.

Let us consider $|E(\mathbb{F}_p)|$. For $x \in \mathbb{F}_p$, half of the x have that $y^2 = x^3 + Ax + B$ is a quadratic residue, giving 2 solutions for y , and the other half are not quadratic residues, giving 0 solutions for y . Thus, we should expect $|E(\mathbb{F}_p)| = p + 1$.

Theorem 19.1 (Hasse's Theorem).

$$||E(\mathbb{F}_p)| - (p + 1)| \leq 2\sqrt{p}.$$

Suppose that $P, Q \in E(\mathbb{F}_p)$. The Elliptic Curve Discrete Log Problem (ECDLP) wants to find $x \in \mathbb{Z}$ such that $xP = Q$, which is well-defined mod $\text{ord}(P)$.

The double and add algorithm aims to compute nP quickly. We compute $P, 2P, 2^2P, 2^3P, \dots$. If $n = n_0 \cdot 2^0 + n_1 \cdot 2^1 + n_2 \cdot 2^2 + \dots + n_r \cdot 2^r$ for $n_i = 0, 1$, then $nP = n_0P + n_1 \cdot 2P + n_2 \cdot 2^2P + \dots + n_r \cdot 2^rP$.

20 Lecture 20

Given $n \in \mathbb{N}$, we want to know how to write n as the sum and difference of powers of 2 such that there are the least nonzero terms. For example,

$$11 = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 = (-1) \cdot 2^0 + 0 \cdot 2^1 + (-1) \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4.$$

Written in binary, we have

$$11 = 1011_2 = 10(-1)0(-1)_2.$$

In general, the binary expansion has $r + 1$ digits, and we may go up to $r + 2$ digits when using differences. We have at most $\frac{r+2}{2}$ nonzero terms if r is even, and $\frac{r+2+1}{2}$ nonzero terms if r is odd.

Suppose that we want to solve ECDLP for $P, Q \in E(\mathbb{F}_p)$. We can try computing $P, 2P, 3P, \dots$. If $N = \text{ord}(P)$, we can instead compute $0, P, 2P, \dots, (n-1)P$ and $Q, Q - nP, Q - 2nP, \dots, Q - (n-1)nP$ and find the overlap, where $n = \lfloor \sqrt{N} \rfloor + 1$.

We discuss the EC DH key exchange. Fix p , $E(\mathbb{F}_p)$, and $P \in E(\mathbb{F}_p)$. For A and B to exchange keys, A will pick n_A , and B will pick n_B . A will then compute $Q_A = n_A P$ and send it to B , and B will compute $Q_B = n_B P$ and send it to A . A and B will agree upon $n_A Q_B = n_B Q_A$.

Based on this, the ECDHP wants to find $n_A n_B P$ given $n_A P$ and $n_B P$.

The EC Elgamal has a fixed p , $E(\mathbb{F}_p)$, and $P \in E(\mathbb{F}_p)$. A 's private key is $n_A \in \mathbb{Z}$, and the public key is $Q_A = n_A P$. Given plaintext $M \in E(\mathbb{F}_p)$, pick a random $k \in \mathbb{Z}$. Let $C_1 = k \cdot P$ and $C_2 = k \cdot Q_A + M$. A will then compute $C - n_A C_1 = C_2 - k \cdot n_A P = C_2 - k \cdot Q_A = M$.

The Menezes-Vanstone Variant of the EC Elgamal has a private key n_A and public key $Q_A = n_A P$. The plaintext is $m = (m_1, m_2) \in \mathbb{F}_p^2$. B chooses a random $k \in \mathbb{Z}$, and has $R = k \cdot P$ and $S = k \cdot Q_A = (x_S, y_S)$. Then, $c_1 = x_S m_1$ and $c_2 = y_S m_2$. B then sends R , c_1 and c_2 . Now, $S = n_A R$, so A can compute $m_1 = x_S^{-1} c_1$ and $m_2 = y_S^{-1} c_2$.

21 Lecture 21

Recall that if $N = pq$ and $p - 1$ is the product of small primes, then we can choose a and compute $a^{n!} \equiv 1 \pmod{p}$ but not $\pmod{q}$, so that $\gcd(a^{n!} - 1, N) = p$.

Let E be $y^2 = x^3 + Ax + B$ over $\mathbb{Z}/N\mathbb{Z}$, and $P \in E(\mathbb{Z}/N\mathbb{Z})$. The idea to replicate the above process is that the order of P in $E(\mathbb{F}_p)$ is not equal to the order of P in $E(\mathbb{F}_q)$.

If $P_1 = (x_1, y_1), P_2 = (x_2, y_2) \in E(\mathbb{Z}/N\mathbb{Z})$, then we define $x(P_1 + P_2) = m^2 - x_1 - x_2$ and $y(P_1 + P_2) = -(m(x - x_1) + y_1)$, where $m = \frac{y_2 - y_1}{x_2 - x_1}$ if $P_1 \neq P_2$, and $\frac{3x_1^2 + A}{2y_1}$ if $P_1 = P_2$. However, $\mathbb{Z}/N\mathbb{Z}$ is not a field, so $E(\mathbb{Z}/N\mathbb{Z})$ does not necessarily form a group. This happens when $\gcd(x_2 - x_1, N) \neq 1$, so it is p, q or N . This means that $P_1 + P_2 = \mathcal{O}$ in $E(\mathbb{F}_p)$, $E(\mathbb{F}_q)$ or both.

We discuss Lanstra's Factorization Algorithm. We set E to be $y^2 = x^3 + Ax + B$, and fix $P \in E(\mathbb{Z}/N\mathbb{Z})$. For $j = 1, 2, \dots$, we compute $j!P$ (using doubling, adding and subtracting). If this fails to compute, we have found $d|N$. If $d \neq N$, we are done, and if $d = N$, we choose a different E and P . If this succeeds to compute, continue to compute $(j + 1)!P$.

One way to choose our E and P is to choose them randomly. We can also choose $P = (a, b) \in (\mathbb{Z}/N\mathbb{Z})^2$ and $A \in \mathbb{Z}/N\mathbb{Z}$. Then for $B = b^2 - a^3 - Aa$, $y^2 = x^3 + Ax + B$ contains P .

22 Lecture 22

In the RSA signature, we can recover D from S , and in DSA, we cannot recover D from S . If we do not need to recover D from S , to boost efficiency of signing, we can try signing on $\text{Hash}(D)$ instead of D . Here, $\text{Hash} : D \mapsto H$, where D has varying length and H has fixed length. Computing Hash should be in linear time, and given H , it is hard to find D with $\text{Hash}(D) = H$. It should also be hard to find D_1 and D_2 with $\text{Hash}(D_1) = \text{Hash}(D_2)$.

Suppose that we have a compression function f , which outputs n bits given $2n$ bits. We use f to construct a Hash function. Given $m_1, m_2, \dots, m_r$, each with at most n bits, we pad with 0s to get n bits. Given an initial value of n bits, we concatenate with m_1 , apply f , concatenate with m_2 , apply f , and so on.

23 Lecture 23

The main idea of bitcoin is to publicly announce transactions, and not trust any establishment. These are placed in a block, which use the hash function to give a block chain. This gives some questions.

- Why should somebody's resources be used to record irrelevant records?
- Whose record should be used in the case that two differ?
- How do you verify authenticity?

The solution to the first question is to offer an incentive of 50 BTC to process a block (with the reward halved every 4 years).

The solution to the second question is Proof-of-Work. The person who solves a hard Hash problem claims the reward. This is to find an element in the preimage of a value under the hash function.

To manage privacy, traditionally, the payer tells the transaction to the bank, who transfers money to the payee, and the public does not know. In bitcoin, the payer is hidden from the public, but the transaction is known. The authenticity is verified using a digital signature (typically ECDSA). We also have to discuss double-spending. If A has 30 BTC and sends 30 BTC to B and C at the same time. Then D might see B get the money before C , and reject the second transaction, while E might see C get the money before B , and reject B 's transaction. We once again use Proof-of-Work to fix this.

24 Lecture 24

We call cryptographic algorithms for one simple task ‘Cryptographic Primitives’. For example, the Hash function, symmetric and asymmetric ciphers, and digital signature are cryptographic primitives.

A Cryptographic Protocol is a set of rules of communication involving several cryptographic primitives for more complex tasks. For example, the key exchange combined with a symmetric cipher.

One protocol is the Commitment Scheme. For example, suppose that A and B are playing a game of rock-paper-scissors (RPS). Then, A will have to choose one of RPS, and fix it. However, A cannot tell it to B . For example, A can write eir choice and place it in a locked box. B then chooses one of RPS , and A gives B the key to the box. B can then check what A chose. When A places eir choice in the box, this is called committing, and when B opens the box, this is called revealing.

The above can be done in a Hash-based method. A will choose a random number r to serve as a key, and will compute $h = \text{Hash}(a, r)$, where a is A ’s choice. B will make eir choice, and A will send a and r to B . In order for A to cheat, A needs to find a collision of Hash, and in order for B to cheat, B needs to find a preimage of h .

This can also be done in a DLP-based method. We fix G , with $|G| = q$ and $g \in G$ a generator. We commit $0 \leq a \leq q - 1$ by publishing $g^a = A$, and we can reveal a later. It is impossible for A to cheat, and B has to solve DLP to cheat. Alternatively, we can choose a random $h = g^x \in G$, with x unknown to both A and B . We commit $0 \leq a \leq q - 1$ by publishing $C = g^a h^b$ for some random b unknown to B . A then reveals a and b . We have that $C = g^{a+bx}$. However, this means that (a, b) and $(a - x, b + 1)$ are both valid. a and b are both unknown, so it is impossible for B to cheat.

We discuss Zero-Knowledge Proofs. The idea is that A wants to prove to B that A knows something without revealing information about it. Further, A wants it to be proven only to B , and somebody else should not be able to be convinced that A knows something. The protocol should satisfy the following:

- Completeness: If A knows something, it is easy for A to answer B ’s questions.
- Soundness: If A does not know something, it is hard for A to answer B ’s questions, and there is a low probability that a random guess is correct.
- Zero Knowledge: B knows nothing except “ A knows something”.

Suppose that you have a colorblind friend, and a red and green ball. You want to prove to your friend that the balls are distinguishable without letting em know which ball is red and which ball is green. One way is for you and your friend to agree on a ball. Your friend then shuffles the balls behind eir back, and asks you to choose the ball again. Ey repeats this multiple times. If you guess each time, the probability of getting it correct every time is 2^{-n} , where n is the number of times the balls were shuffled. Thus, repeating it enough times, your friend can be convinced that the balls are distinguishable without knowing the color of either.

Suppose you have a circular hallway that is blocked at one point by a door. To open the door, you need to know a secret word. A wants to prove to B that ey knows the word

without telling B the word. A chooses a random side to enter, and B does not know which side. B chooses a side, and asks A to come out on that side. This is easy if A knows the secret word. After n tries, the probability of A succeeding by guesswork is 2^{-n} . It is possible for B to have A enter one side and exit the other. However, B can record this, and can use the video to convince others that A knows the secret word. If B records the earlier process, ey will not be able to convince others that A knows the secret word, as A and B could be collaborating.

Suppose that A wants to prove to B that ey knows the answer to $A = g^a$. Ey will choose a random r , and tell B $R = g^r$. B can ask A to reveal either r or $a + r$, and can check if $R = g^r$ in the first case, or if $AR = g^{a+r}$ in the second case.

25 Lecture 25

The building block of a classical computer is a bit, which takes values 0 or 1. The building block of a quantum computer is called a qubit, which has basis states $|0\rangle$ and $|1\rangle$, and takes values $\alpha|0\rangle + \beta|1\rangle$, where $\alpha, \beta \in \mathbb{C}$ satisfy $|\alpha|^2 + |\beta|^2 = 1$. Here, the states $|0\rangle$ and $|1\rangle$ exist simultaneously, with probability $|\alpha|^2$ and $|\beta|^2$ respectively. An n -component basis state is

$$|0, 0, \dots, 0\rangle, |1, 0, \dots, 0\rangle, |0, 1, \dots, 0\rangle, |1, 1, \dots, 0\rangle, \dots, |1, 1, \dots, 1\rangle,$$

and a n -qubit takes values

$$\alpha_0|0, 0, \dots, 0\rangle + \dots + \alpha_{2^n-1}|1, 1, \dots, 1\rangle,$$

with $|\alpha_0|^2 + \dots + |\alpha_{2^n-1}|^2 = 1$. With an n -qubit, we can simultaneously represent 2^n numbers, which allows parallel computing. The algorithm called Shor's Algorithm can solve RSA, DLP, and ECDLP in polynomial time if we have a quantum computer of large enough size.

A lattice in $\mathbb{R}^m$ is $L := v_1\mathbb{Z} + v_2\mathbb{Z} + \dots + v_n\mathbb{Z}$ for $v_1, v_2, \dots, v_n$ linearly independent (in particular, $n \leq m$). We write $\dim L = \text{rank } L = n$, and we call $\{v_1, v_2, \dots, v_n\}$ a basis of L .

Notice that these bases are not necessarily unique. For example, $(0, 1)\mathbb{Z} + (1/2, 1)\mathbb{Z} = (0, 1)\mathbb{Z} + (1/2, 0)\mathbb{Z}$ in $\mathbb{R}^2$.

Proposition 25.1. *Any two bases of a lattice differ by a matrix with integer coefficients and determinant ± 1 .*

Proof. Let $\{v_1, \dots, v_n\}$ and $\{w_1, \dots, w_n\}$ be two bases of a lattice $L \subseteq \mathbb{R}^m$. Consider

$$\begin{pmatrix} | & \dots & | \\ v_1 & \dots & v_n \\ | & \dots & | \end{pmatrix} = \begin{pmatrix} | & \dots & | \\ w_1 & \dots & w_n \\ | & \dots & | \end{pmatrix} \cdot A,$$

where the first two matrices are $m \times n$, and A is an $n \times n$ matrix. Then, $v_i = \sum_{j=1}^n w_j a_{j,i}$. Now, $v_i \in L = w_1\mathbb{Z} + \dots + w_n\mathbb{Z}$, so $a_{i,j} \in \mathbb{Z}$. In particular, $\det A \in \mathbb{Z}$. Now, $\{v_1, \dots, v_n\}$ and $\{w_1, \dots, w_n\}$ are bases of an n -dimensional vector space in $\mathbb{R}^m$, so $\det A \neq 0$. Thus, A^{-1} exists, and

$$\begin{pmatrix} | & \dots & | \\ v_1 & \dots & v_n \\ | & \dots & | \end{pmatrix} \cdot A^{-1} = \begin{pmatrix} | & \dots & | \\ w_1 & \dots & w_n \\ | & \dots & | \end{pmatrix}.$$

Thus, the coefficients of A^{-1} are integers. Thus, $\det A^{-1}$ is an integer. Now, $1 = \det A \det A^{-1}$, so $\det A = \pm 1$. ■

We write $\text{GL}_n(\mathbb{Z})$ for the set of $n \times n$ matrices with integer coefficients and determinant ± 1 .

26 Lecture 26

Let $L = v_1\mathbb{Z} + \cdots + v_n\mathbb{Z} \subseteq \mathbb{R}^m$ be a lattice. The fundamental domain $\mathcal{F}$ of L is

$$\{\alpha_1 v_1 + \cdots + \alpha_n v_n : 0 \leq \alpha_i < 1\}.$$

If $n = m$, then $\mathbb{R}^m = \mathcal{F} + L$ (i.e. the fundamental domain translated by the lattice).

If L is full rank, then the volume of $\mathcal{F}$ is $|\det(v_1 \cdots v_n)|$. More generally, if $n \neq m$, then the volume of $\mathcal{F}$ is $\sqrt{\det(\langle v_i, v_j \rangle)}$. To see the second statement, let $w_1, \dots, w_n$ be the orthogonal basis after the Gram-Schmidt process to $v_1, \dots, v_n$. Then, the volume of $\mathcal{F}$ is

$$\sqrt{\|w_1\|^2 \cdots \|w_n\|^2} = \sqrt{\det \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix} (w_1 \cdots w_n)} = \sqrt{\det S^\top \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix} (v_1 \cdots v_n) S},$$

where S is an upper triangular change of basis matrix with diagonal entries 1. Thus, $\det S = \det S^\top = 1$.

In particular, the volume of $\mathcal{F}$ is independent of the choice of basis of L , as change of bases matrices have determinant ± 1 .

Proposition 26.1. *The volume of $\mathcal{F}$ is at most $\|v_1\| \cdots \|v_n\|$.*

Definition 26.2. The Hadamard ratio $\mathcal{H}$ for $\{v_1, \dots, v_n\}$ is

$$\sqrt[n]{\frac{\det L}{\|v_1\| \cdots \|v_n\|}} \leq 1,$$

where $\det L$ denotes the volume of $\mathcal{F}$.

The shortest vector problem (SVP) aims to find $v \in L$ minimizing $\|v\|$. The closest vector problem (CVP) aims to find $v \in L$ minimizing $\|v - w\|$ for $w \in \mathbb{R}^m \setminus L$. Solving these is easy if we have an orthogonal basis $v_1, \dots, v_n$ of L .

27 Lecture 27

We discuss the Goldreich-Goldwasser-Halevi (GGH) cryptosystem. A will have a private key $\{v_1, \dots, v_n\}$ with $\mathcal{H}_v \sim 1$. The public key will be a basis $\{w_1, \dots, w_n\}$ of the same lattice, such that $\mathcal{H}_w \sim 0$. Write $(w_1 \ \dots \ w_n) = (v_1 \ \dots \ v_n) U$. For the plaintext $m \in \{0, 1\}^n$, choose a small vector $r \in \mathbb{R}^m$. The ciphertext is $c = \sum_{i=1}^n m_i w_i + r \notin L$. A can then use $v_1, \dots, v_n$ to find the closest vector to c in L , and write it in terms of the w_i .

To find a “nice” basis for L , we mimic the Gram-Schmidt process. For example, suppose $L = v_1\mathbb{Z} + v_2\mathbb{Z}$, and $\|v_1\| \leq \|v_2\|$. We replace v_2 with $v_2 - rv_1$, where

$$r = \left\lfloor \frac{\langle v_2, v_1 \rangle}{\langle v_1, v_1 \rangle} \right\rfloor.$$

We then repeat this with our new v_2 . This process must terminate, as there cannot be an accumulation point.